M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_2

2. Fundamental Elements of Browser Extensions

Matt Frisbie[1]

(1) California, CA, USA

As with most software platforms, browser extensions can be conceptually divided into a handful of discrete pieces. Developers new to the world of browser extensions may find the idiosyncrasies of these pieces tricky to internalize, as some of the behavior is redundant or not intuitive. Understanding how the pieces of browser extensions fit together is critical for becoming an expert at browser extension development.

> **Note** This chapter will individually cover each piece of browser extensions at a high level and without diving into too much code. Subsequent chapters will probe each of these pieces in depth.

The Browser Model

Before exploring the various elements of browser extensions, let's first begin by reviewing the default behavior of a browser. Consider the following diagram of a web browser with an open tab (Figure 2-1).

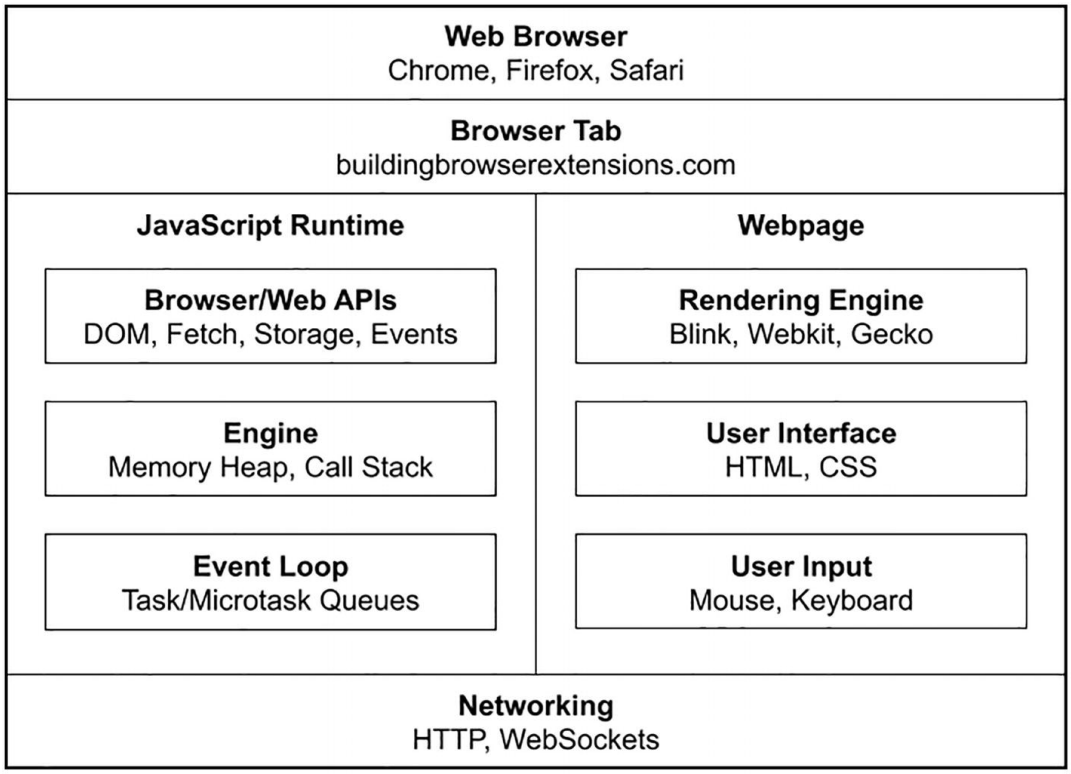

Figure 2-1 Simplified model of the web browser

> **Note** A basic understanding of how a web page is rendered, how JavaScript executes, and how the browser performs network requests is a prerequisite for browser extension development. If these topics are unfamiliar to you, an excellent list of resources can be found at **https://developer.mozilla.org/en-US/docs/Web/Guide/Introduction_to_Web_development**

This diagram illustrates the high-level concepts at play when a web browser has a single tab directed to a URL. The web page renders in the browser's primary user interface and has its own JavaScript runtime. Both the web page itself and the scripts within it make subsequent network requests after the initial HTML loads: the page head will load assets like CSS, images, and additional scripts, and the scripts can execute additional requests to load assets and send and receive data.

Browser Tabs

Let's extend the previous diagram to show when multiple tabs are open in a browser (Figure 2-2).

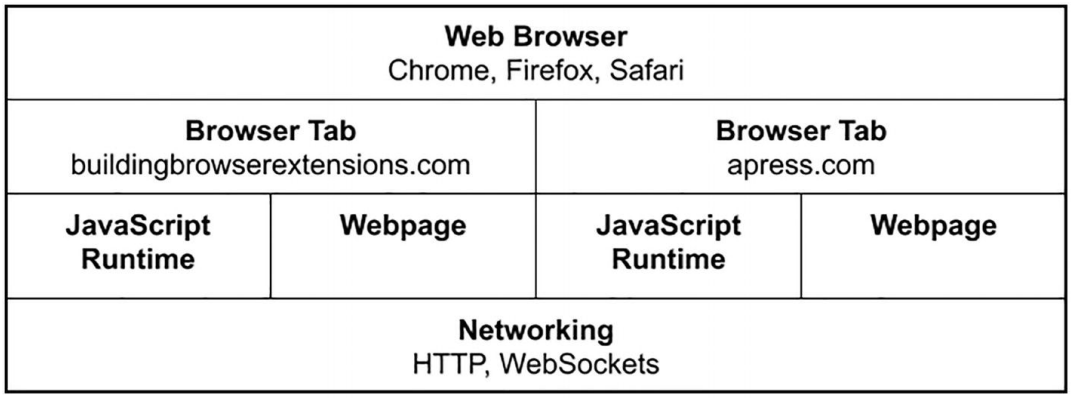

Figure 2-2 Web browser with multiple tabs

Browsers effectively sandbox different tabs, providing each with their own document object, JavaScript runtime, memory, and depending on the browser, their own system thread and process. Of course, there are a myriad of critical considerations when it comes to matching origins. Although each tab will be afforded separate runtimes, two separate tabs with matching origins are able to share resources. To name a few

- Cookies
- LocalStorage
- IndexedDB
- Shared Workers
- Service Workers
- HTTP Cache

Same-Origin Policy

Origins are also a consideration when applying the Same-Origin Policy (SOP). In brief, the SOP is a set of security policies implemented by all browsers that control access to data between web applications. When two web applications have different origins, the browser automatically applies a set of restrictions to protect potentially sensitive information from an untrusted origin. Web developers typically encounter the SOP in three places: when sending network requests to foreign origins, when executing scripts from a different origin, and when accessing storage APIs from different origins.

The Browser Extension Model

From the perspective of the web page, browser extensions can be thought of as an invisible supplemental entity. They execute JavaScript in their own runtime, they render pages in their own sandboxed contexts, and they have access to their own separate set of APIs. The web page will have no idea browser extensions are installed or what it they are doing.

> **Note** Content scripts are an important exception to this, as they allow for both the web page and the extension to access the DOM and some shared resources, but as you will see in the *Content Scripts* chapter, they are subjected to some extension-specific sandboxing of their own.

Consider the diagram of a browser with an extension installed as in Figure 2-3.

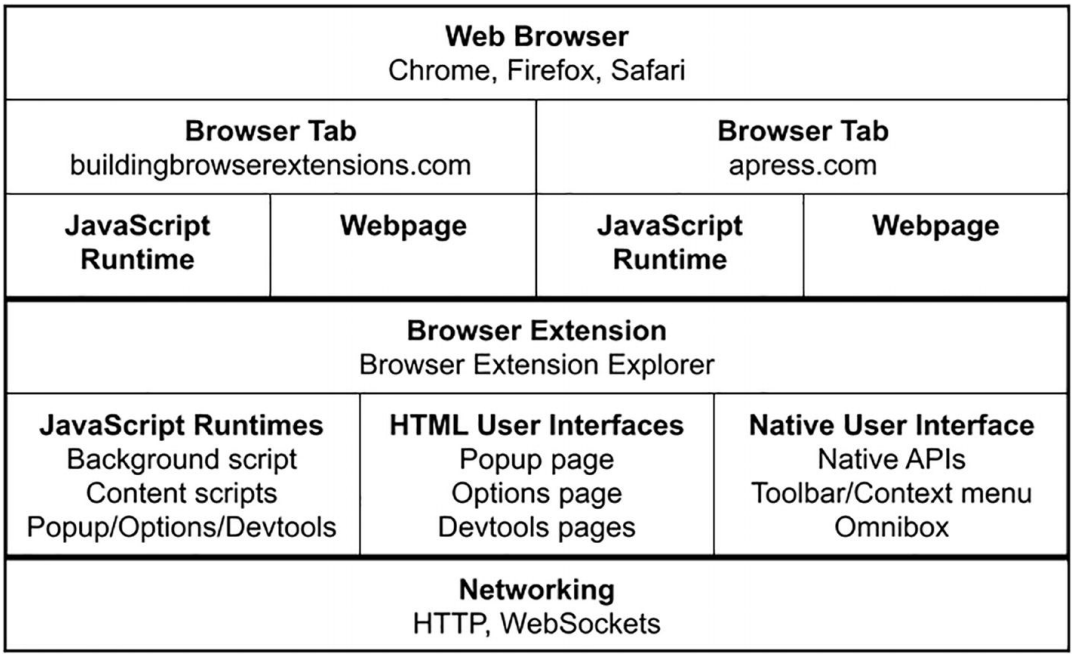

Figure 2-3 Web browser with multiple tabs and an extension installed

Let's examine each of the important concepts shown in this diagram.

Independent JavaScript Pages and Runtimes

Once installed, browser extensions can offer multiple custom HTML user interfaces in several places: the popup page, the options page, and the

devtools pages. These pages behave like normal web pages in nearly every way:

- Each page is rendered in the same way as a normal web page, with load events and a conventional document object.
- Each page is provided a dedicated native browser container and completely separated from all other web pages.
- Each interface is provided its own JavaScript runtime.

Background scripts and content scripts are provided their own runtime, but they do not exist as a standalone user interface.

> **Note** The details of popup, options, devtools, background script, and content script are covered later in this chapter.

Native APIs and User Interfaces

Browser extensions have access to a suite of native browser APIs that allow them to control the behavior, appearance, and data of the browser. Some examples

- Tabs API
- Bookmarks API
- Downloads API
- Browser History API
- Browsing Data API

> **Note** These APIs will be covered in depth in the *Extension and Browser APIs* chapter.

Furthermore, browser extensions are afforded several ways to integrate with the browser user interface itself. This includes the ability to manage how the toolbar icon appears and behaves, specifying behavior for the URL bar omnibox interface, and defining what should appear in the extension context menu when it is opened.

Tab and Domain Access

Browser extensions are not restricted to a single domain or tab – they live in the browser itself. When given the proper permissions, browser extensions are capable of interacting with multiple tabs on multiple domains. This means that a browser extension is able to inspect and manage a portion or the entirety of a browser's open pages.

Browser extensions are also not required to use a web page at all. Extensions are adequately equipped to exist as standalone pieces of software that live only in the browser itself. Of course, some aspects of browser extensions like content scripts and devtools interfaces are only useful when manipulating web pages.

Observing and Intercepting Network Requests

Normally, network requests dispatched from a web page are handed off directly to the browser. Browser extensions can be given permission to act as a network request shim, where requests dispatched to the page will be preprocessed by the extension before being passed off to the browser. This means that the extension can add rules for conditionally modifying requests, or even blocking them outright. It also means that the extension can see a stream of traffic in an out of the page in real time.

> **Note** The mechanics of modifying network requests are being significantly changed between manifest v2 and v3. This is covered in depth in the *Networking* chapter.

Elements of Browser Extensions

The previous section covered browser extensions as a holistic entity. This is useful when trying to get a grasp on the concept as a whole. However, as alluded to in the introduction to this chapter, browser extensions exist as a collection of semi-independent elements. This section will examine each of these elements at a high level.

Extension Manifest

The manifest is the rulebook for the browser extension. Every extension is required to provide this manifest as JSON data in a `manifest.json` file. Some examples of what is contained in the manifest

- Public information such as the extension name, description, semantic version, icons, and author
- Pointers to entrypoint files for background scripts, popup pages, options pages, devtools pages, and content scripts
- Requirements and configurations the extension needs to properly operate, such as permissions, content security policies, cross-origin policies, and minimum browser versions
- Pattern-match rule sets for managing network requests, enabled domains, and resources the extension wishes to inject into the page via content script
- Miscellaneous extension-specific options like enabling offline and incognito and keyboard shortcuts

> **Note** The specifics of how to write a `manifest.json` file can be found in the *Extension Manifests* chapter.

Manifest v2 and v3

The world of browser extensions is currently in a transitional period between two versions of the `manifest.json` file: version 2 (v2) and version 3 (v3). The new manifest v3 significantly alters how extensions execute, and it changes the APIs that they can access. Google Chrome is spearheading the transition to manifest v3; due to their market dominance, other browser vendors are for the most part following in turn.

The transition to manifest v3 will be a major theme in this book. The changes it makes are highly controversial in the browser extension developer community, and they broadly impact a number of extremely popular browser extensions.

Background Scripts

The primary purpose of background scripts is to handle browser events. These events might be extension lifecycle events such as an install or uninstall, or they might be browser events like navigating to a web page or adding a new bookmark. Background scripts can be defined in JavaScript files, or as a single HTML file that runs the background scripts via one or more `<script>` tags in a headless page.

> **Tip** Most simple browser extensions are adequately served by a single background script.

Although background scripts are provided their own JavaScript runtime, they do not exist in isolation. They can access the WebExtensions APIs and therefore are capable of performing actions such as exchanging messages with other parts of the extension, exchanging messages with other extensions, or programmatically injecting content scripts into a page.

Background scripts undergo a significant transformation between manifest v2 and manifest v3. In manifest v2, the background script had the option of being either "persistent," meaning the background script is initialized exactly once and lives in memory until the browser is closed, or nonpersistent, where the background script exists as an "event page" that is initialized on demand whenever a relevant browser event occurs. In manifest v3, background scripts exist as service workers, which largely replicate the behavior of the manifest v2 nonpersistent background scripts.

> **Note** The *Background Scripts* chapter includes more details on background scripts.

The Popup Page

The popup page is a native browser container that browser extensions can use to display a custom user interface. The popup page behaves as a dialog

box that "pops" over the web page when the user clicks the extension toolbar button. The popup page will always appear directly below the toolbar. Because it can be quickly accessed and can show over any web page, the popup page typically contains content that users need easy access to. An example of a popup page is shown in Figure 2-4.

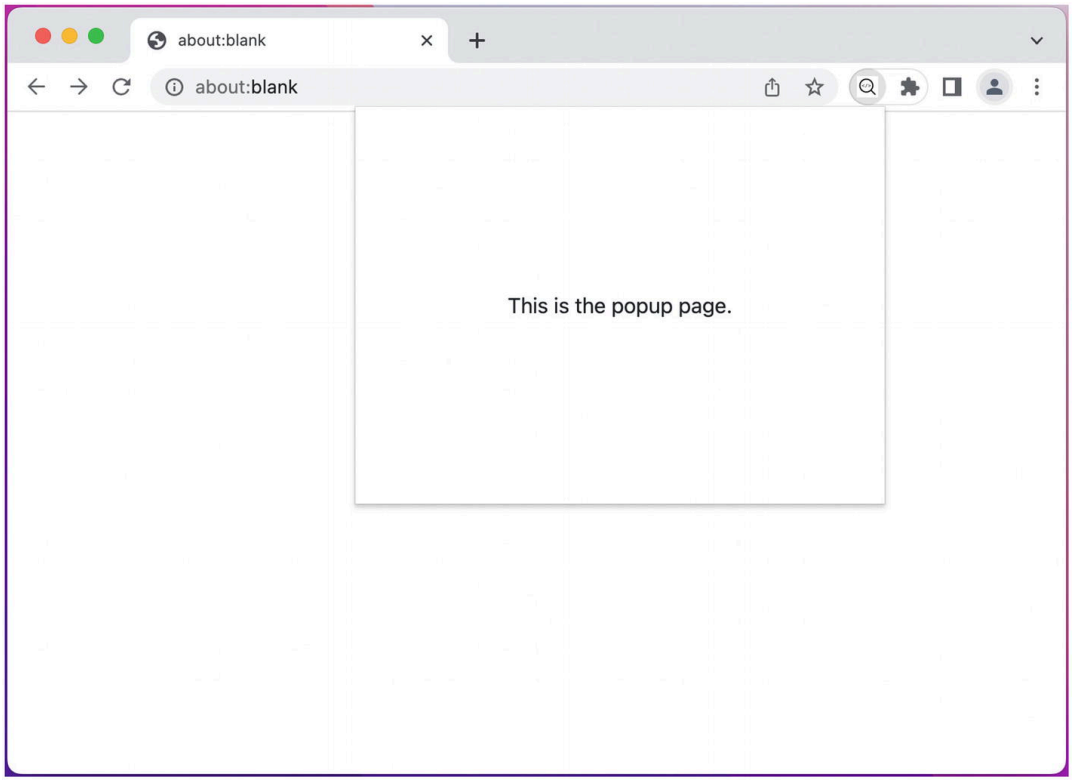

Figure 2-4 Simple popup page

Popup pages are rendered just like regular web pages, but their dialog-like nature means they are disposable: the popup will be freshly rendered each time the popup is opened, and unloaded when the popup is closed. Like background scripts, popup pages can access the WebExtensions API, meaning they have the same set of capabilities.

> **Note** The popup page cannot be opened programmatically, it must be triggered by a toolbar click or similar privileged browser action. This is to prevent extensions from abusing this ability and frequently or opportunistically opening their popup pages, which would be problematic for the user experience.

The Options Page

The options page is a native browser container that browser extensions can use to display a custom user interface. The options page behaves as a standalone web page that opens when the user clicks “Options” in the extension toolbar context menu. An example of opening an options page is shown in Figures 2-5 and 2-6.

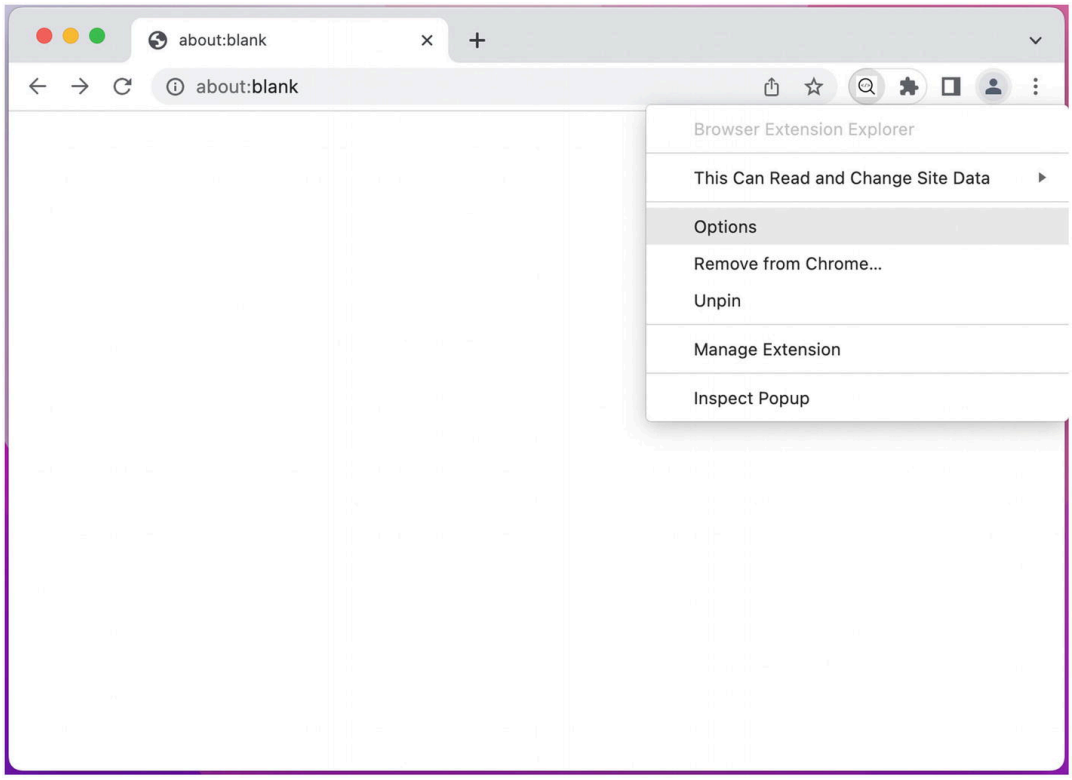

Figure 2-5 Selecting “Options” from the extension context menu

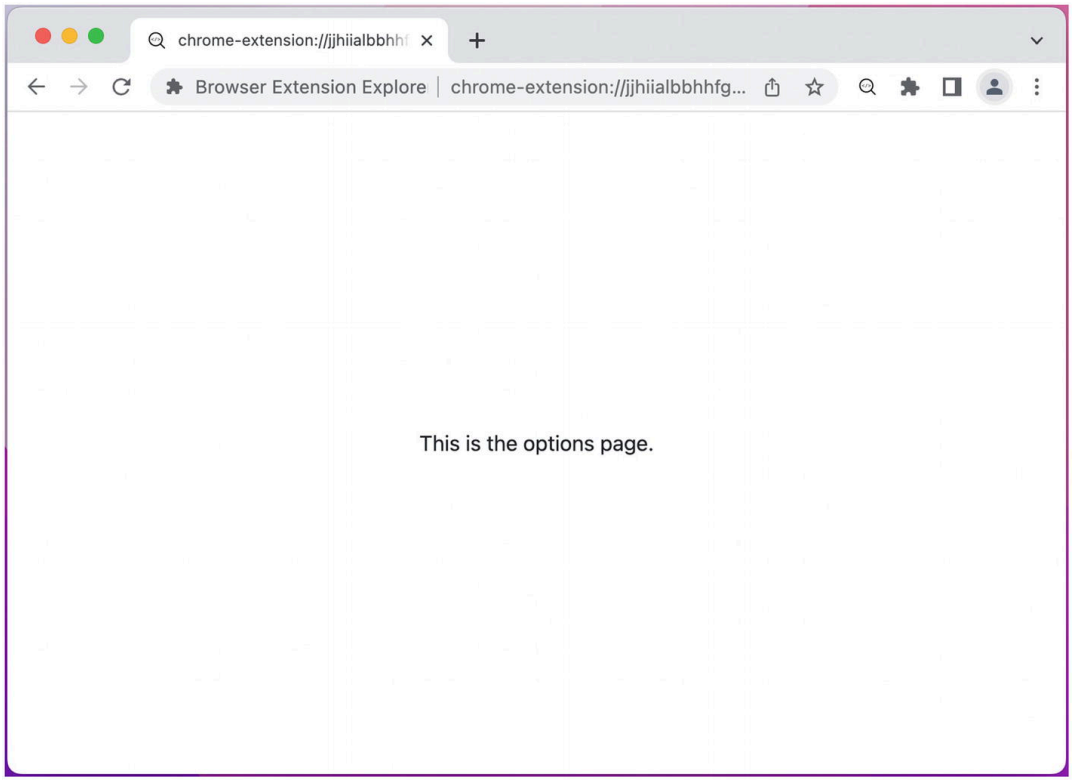

Figure 2-6 Simple options page

The “options” name is misleading: this page is not restricted to just showing options for the extension. Like the popup page, this is a fully featured web page with access to the WebExtensions API, meaning you are capable of using it as a full web application.

> **Note** Refer to the *Popup and Options Pages* chapter for more details on how these pages work and how to use them.

Content Scripts

The term *content script* broadly refers to any content that is injected into a web page. JavaScript can either be injected declaratively in the manifest, or programmatically from an extension page or background script via the WebExtensions API. This content can be JavaScript, CSS, or both. The JavaScript is sandboxed in its own separate runtime, meaning it cannot read JavaScript variables or properties in the primary web page runtime, but it still shares access to the same DOM as the web page itself. This means that content scripts are fully capable of

reading and writing the page, enabling things like in-page widgets or full integration with the web page.

Content scripts have limited access to the WebExtensions API, so they are incapable of many actions that are possible in the popup page, options page, or background script. They can, however, exchanges messages with other extension elements like background scripts. Therefore, content scripts can still indirectly use the WebExtensions API by delegating actions to a background script.

> **Note** Refer to the *Content Scripts* chapter for more details on how content scripts work and how to use them.

Devtools Panels and Sidebars

Devtools panel and sidebar pages are native browser containers that browser extensions can use to display a custom user interface. The devtools interfaces behave as panes nested in the native browser's developer tools when the user selects their corresponding tabs in the developer tools interface, shown in Figures 2-7 and 2-8.

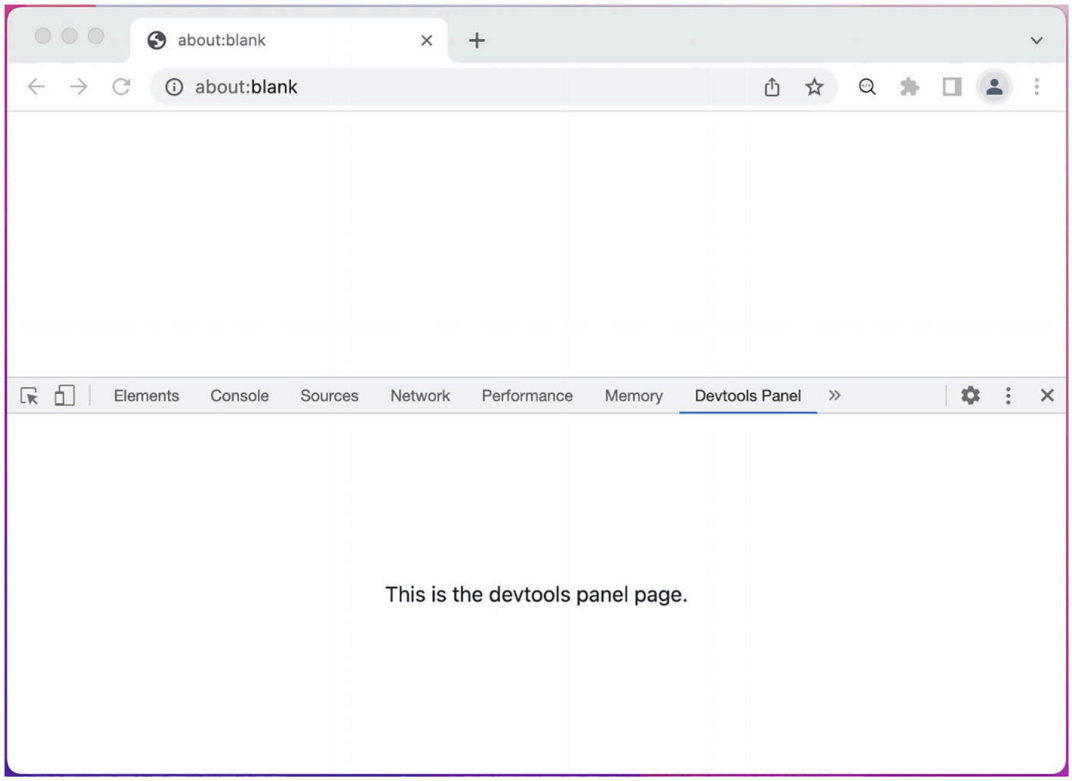

Figure 2-7 Simple devtools panel page

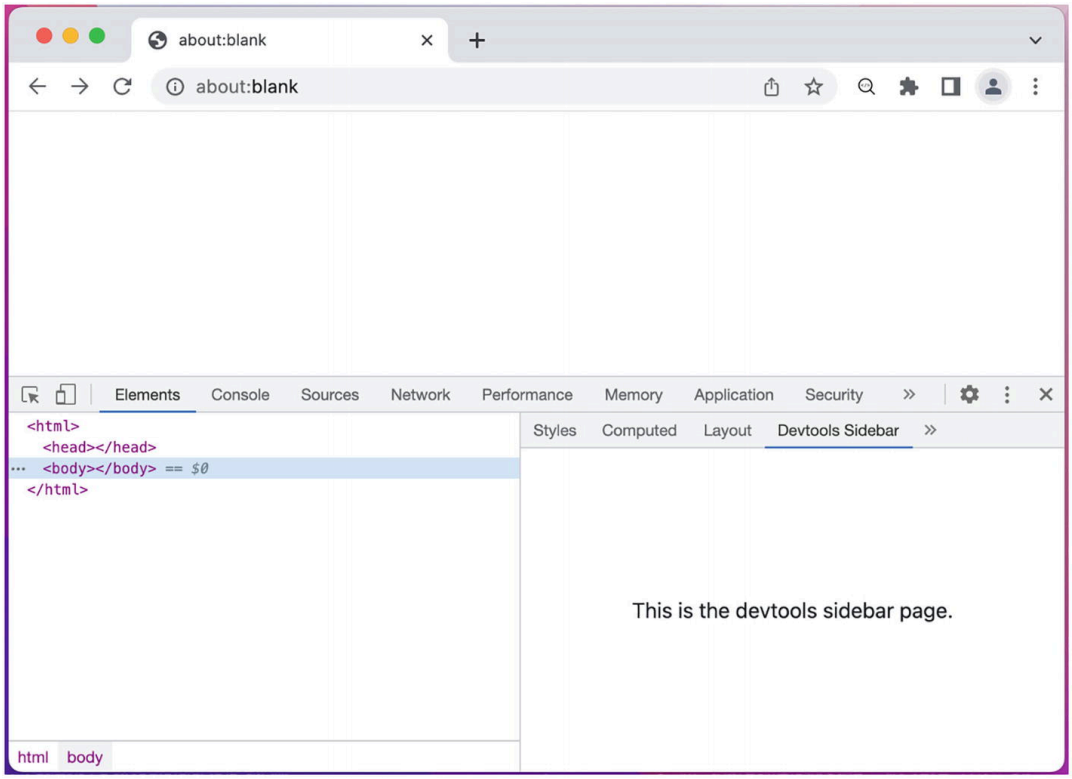

Figure 2-8 Simple devtools sidebar page

Like the popup page, this is a fully featured web page, but like content scripts, it has limited access to the WebExtensions API. However, it is provided access to the additional Devtools API that can be used to perform actions like inspecting the page and observing the page's network traffic.

> **Note**Refer to the *Devtools Pages* chapter for more details on how these pages work and how to use them.

Extension Elements in Action

To aid in understanding how all these elements fit together, let's examine how these pieces fit into a handful of popular browser extensions. Based on what we know about how the various elements of browser extensions behave, we can understand how the extension is accomplishing these user interfaces even without having access to the codebases.

Honey

Honey is a browser extension that can detect when the user is checking out on an ecommerce website and automatically apply coupon codes (Figure 2-9).

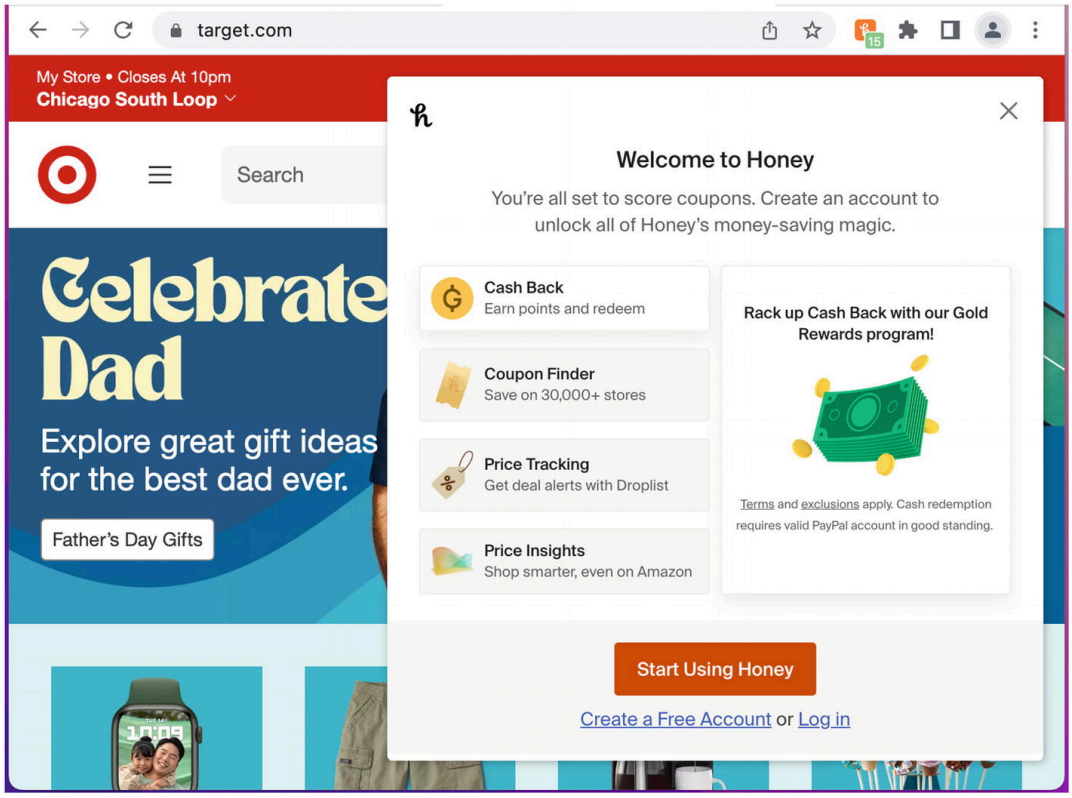

Figure 2-9 Honey browser extension showing content script widget and toolbar icon with badge

Here, Honey is rendering a widget over the page using a content script. It also uses a content script to programmatically submit coupon codes when the user is in the checkout flow.

Furthermore, note that the toolbar icon is displaying a badge containing "15," which is Honey telling us there are 15 possible coupon codes to try on this domain. Honey knows this because it can assess what domain the user is visiting and checking that against the Honey database of matching coupon codes. The extension is then able to use its ability to dynamically set what the toolbar icon shows and render the Honey logo with the "15" badge (Figure 2-10).

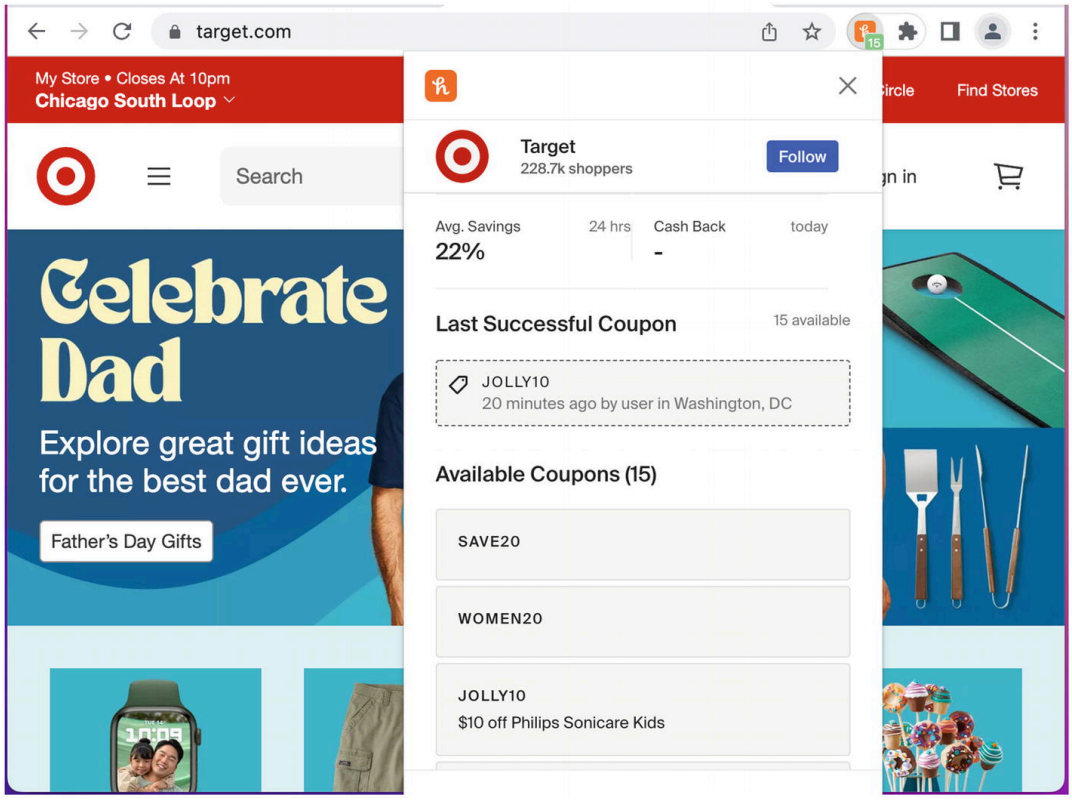

Figure 2-10 Honey browser extension showing the popup page

The above screenshot shows the popup view in Honey. Honey does not rely on this as a primary user interface, as it is not possible to programmatically open the popup window. The extension needs to automatically display content to the user when it detects that it can submit coupon codes, and the extension relies on widgets powered by content scripts to accomplish this.

LastPass

LastPass is a browser extension that can securely store and autofill a user's passwords, as well as record new passwords when they are submitted (Figure 2-11).

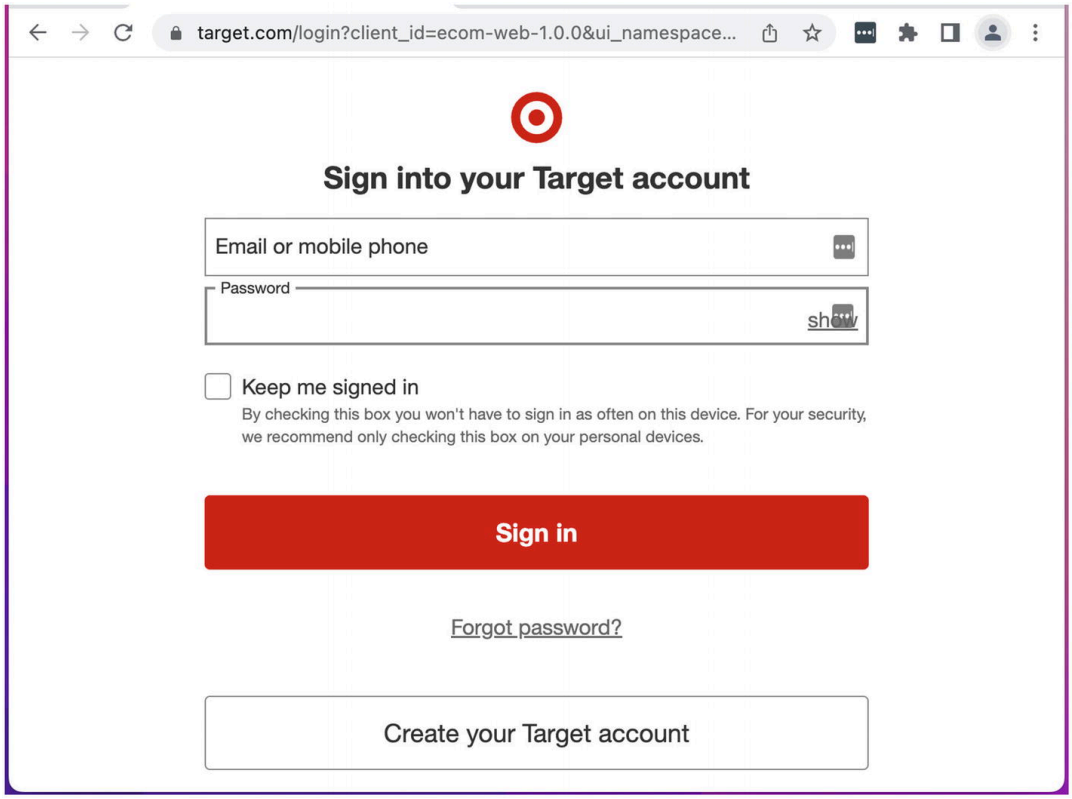

Figure 2-11 LastPass browser extension with content script widget buttons

LastPass uses a content script to find and fill authentication form fields, as well as to render widgets for managing passwords from within the form fields. LastPass also watches traffic in and out of the browser for requests that set or update passwords and records the request payload in your encrypted account credential list (Figure 2-12).

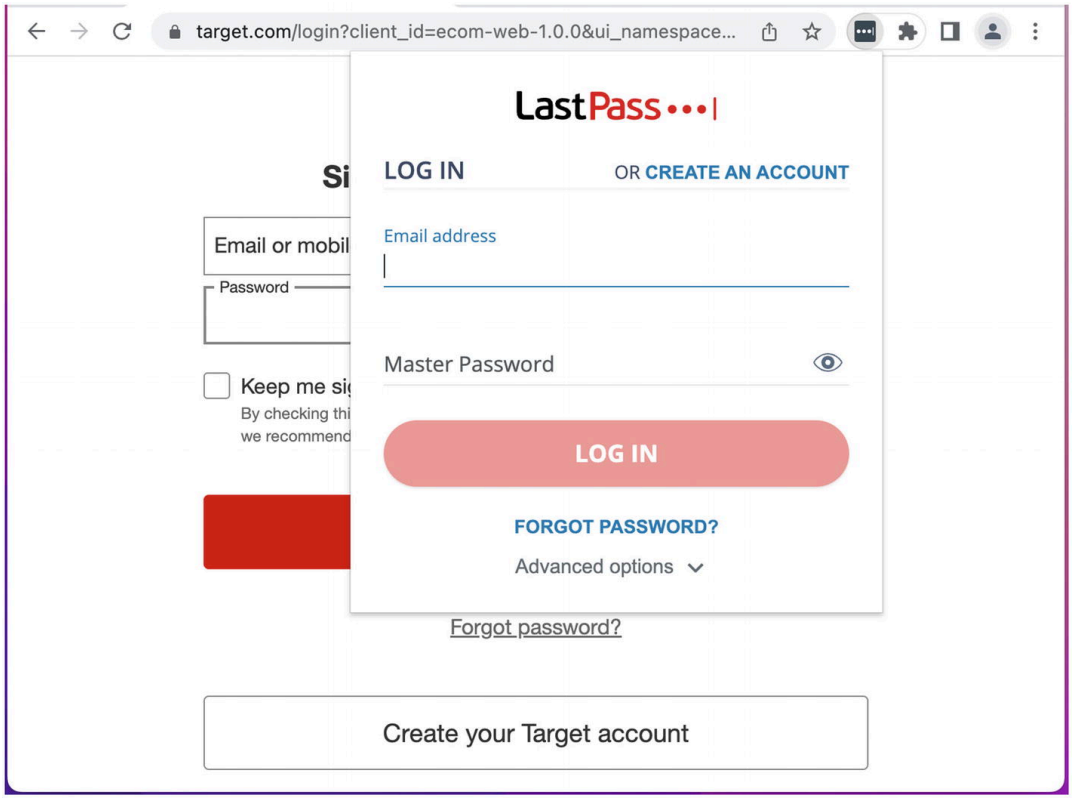

Figure 2-12 LastPass browser extension showing popup page

Of course, form detection will never be perfect, so LastPass offers all the same password utilities from the popup window to account for scenarios where the content script widget does not render (Figure 2-13).

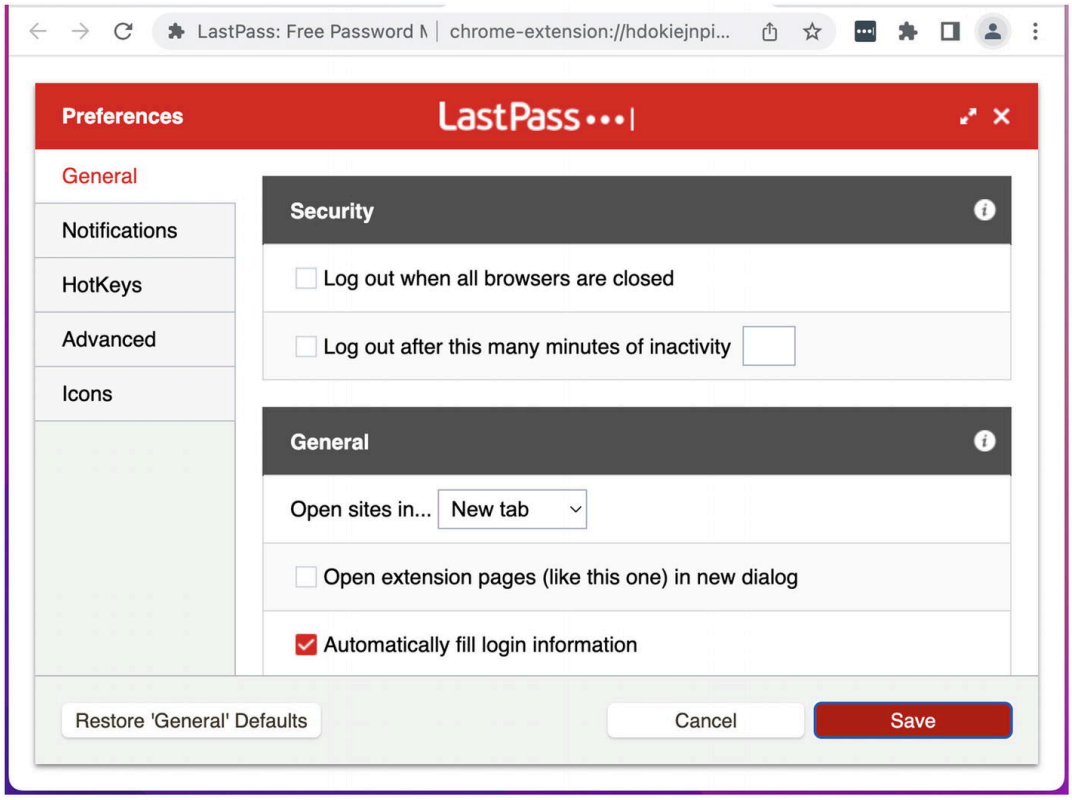

Figure 2-13 LastPass browser extension showing options page

In addition to managing passwords, LastPass has a broad suite of security tools and allows for intricate customization of its behavior. The browser extension exposes these settings in the extension options page, shown above.

Grammarly

Grammarly is a browser extension that observes the text a user types into the browser and automatically offers suggestions on how to fix or improve it (Figures 2-14 and 2-15).

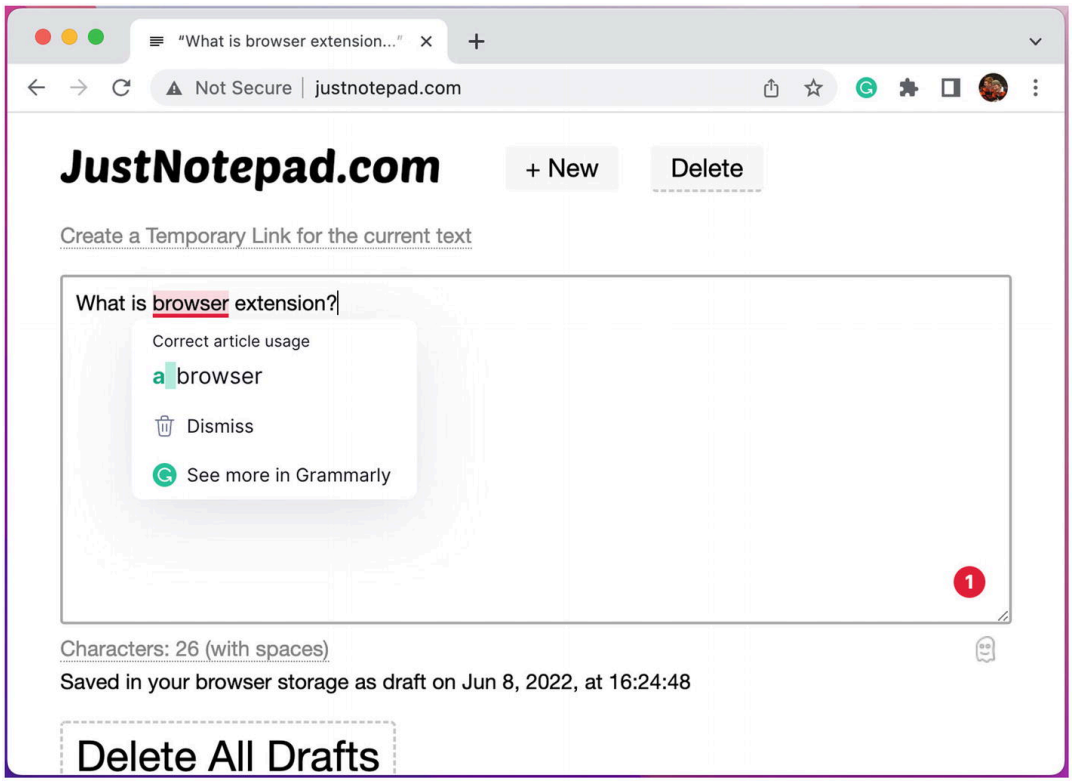

Figure 2-14 Grammarly browser extension showing the content script widget

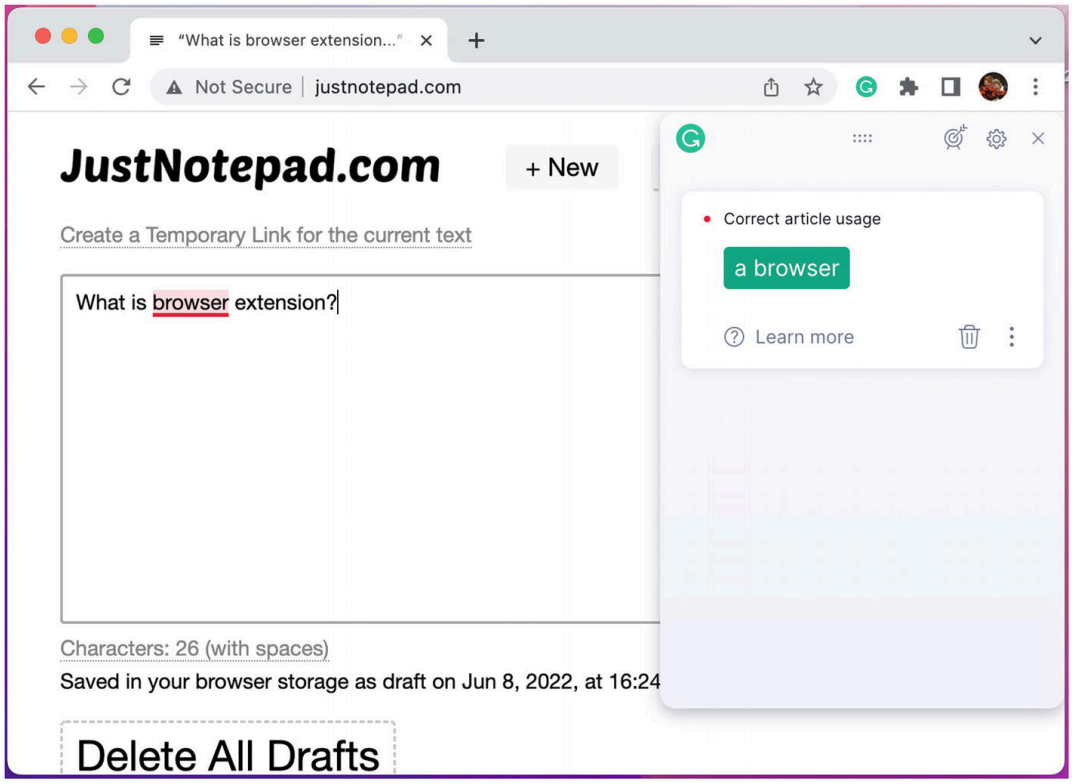

Figure 2-15 Grammarly browser extension showing the collated content script widget

Similar to how LastPass detects authentication form fields, Grammarly uses a content script to intelligently detect when the user is typing content into the page, parses that content, and renders suggestions on top of that content. The extension is also able to collate all active suggestions into the unified content script widget (Figure 2-16).

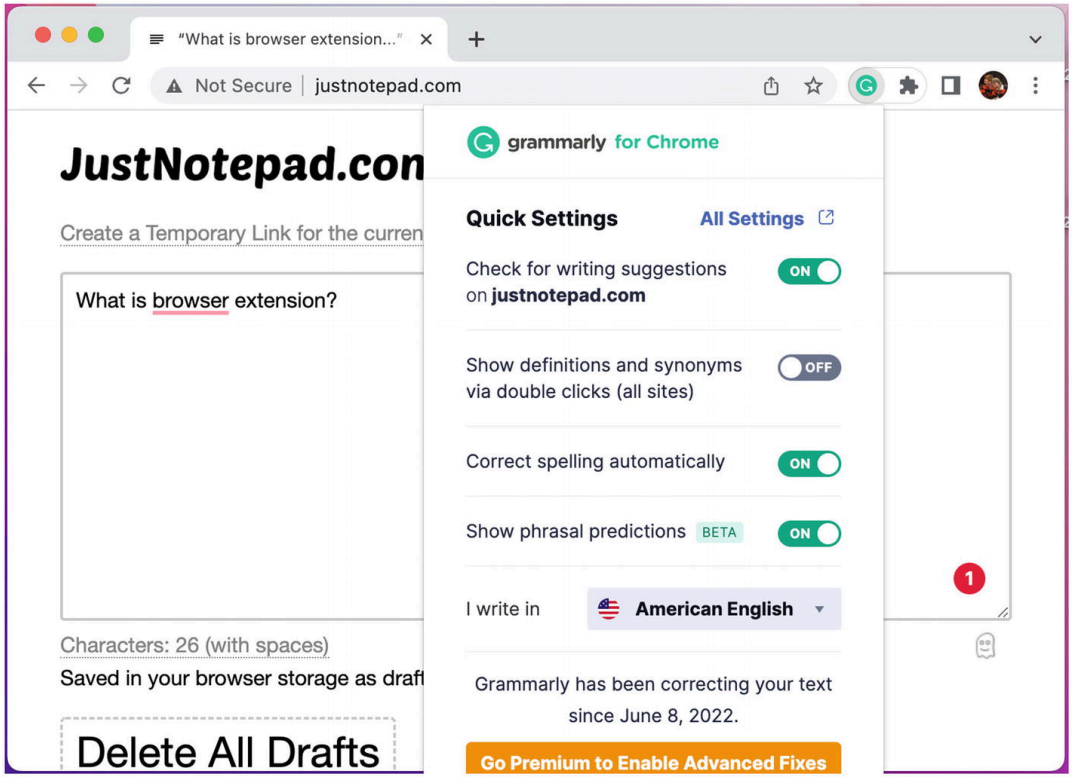

Figure 2-16 Grammarly browser extension showing the content script widget

Grammarly also offers toggles for extension settings in the popup page.

React Developer Tools

React Developer Tools is a browser extension that allows the user to understand how a React application is rendering in the page (Figure 2-17).

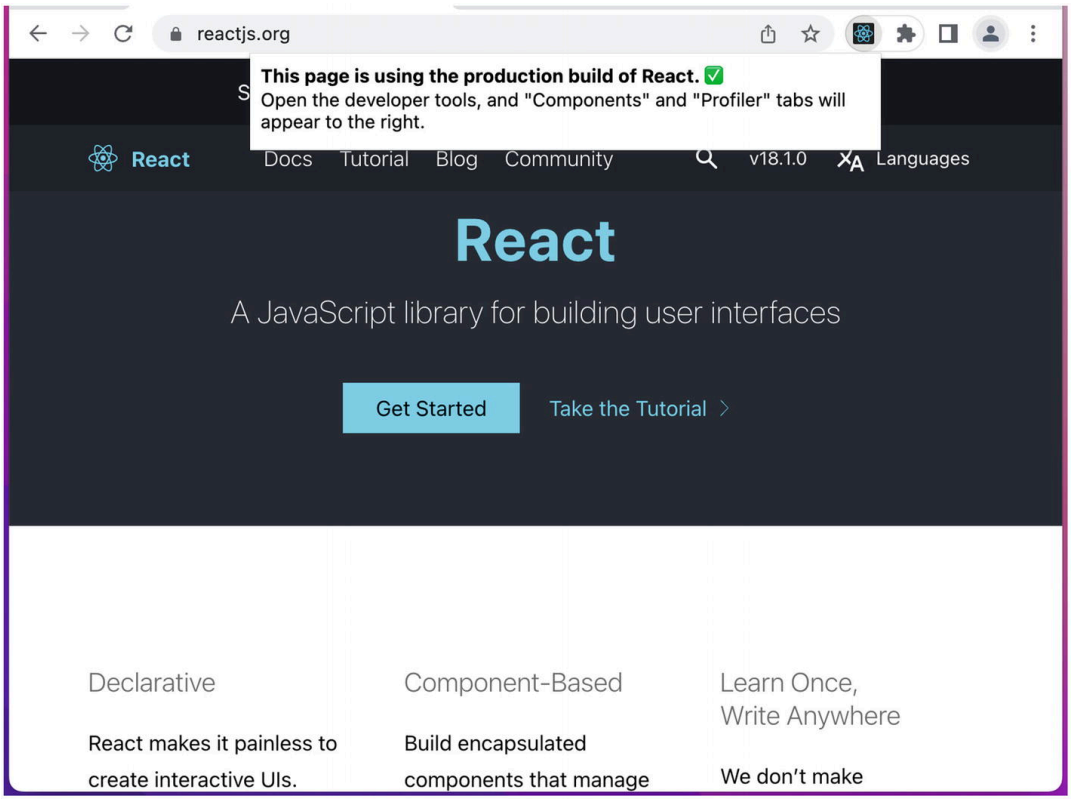

Figure 2-17 React Developer Tools browser extension showing the popup widget

The browser extension can analyze the web page to detect if it is a React application. If it is, the toolbar icon will indicate as much, and the popup page informs the user what type of React application is present (Figure 2-18).

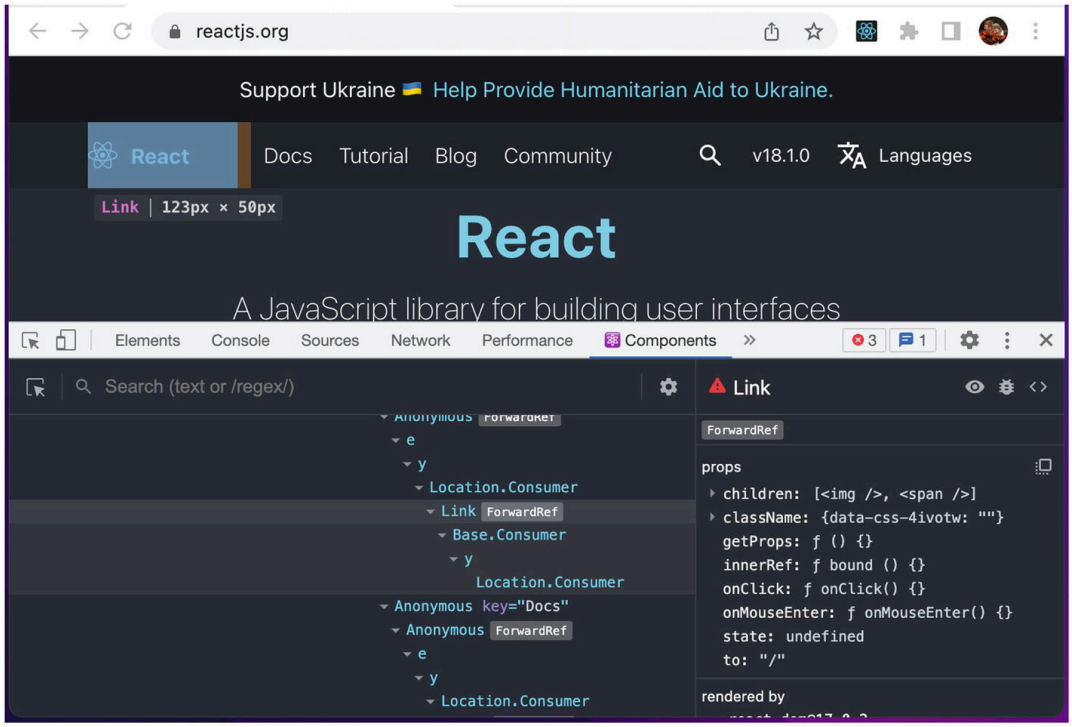

Figure 2-18 React Developer Tools browser extension showing a devtools panel page with element highlighted

The extension uses the Devtools API to create a custom panel in the browser's developer tools interface. It parses the page content and unpacks it into a browseable component tree inside the panel page.

Summary

As the name suggests, browser extensions are more of an extension of the browser rather than the page. The WebExtensions API allows extensions to perform actions that web pages cannot normally accomplish.

Browser extensions are composed of a collection of elements. The only required element is the manifest. Optional elements include background scripts, the popup page, the options page, devtools pages, and content scripts.

The next chapter will take you through a crash course covering how to build and install a basic browser extension.